PROBLEM STATEMENT NO. 183 · CATEGORY: MODEL MAKING

Multi-Document RAG with Citations

An AI-powered system for reliable, source-verified question answering across multiple documents

PRESENTED BY

Honey Soni (24, ADT23SOCB0444)
Roshan Warade (35, ADT23SOCB0883)
Virendra Mahajan (41, ADT23SOCB1322)
Poornima Rayavarapu (31, ADT23SOCB0727)
Sujal Jadhavar (37 , ADT23SOCB1167)

THE PROBLEM

Answers live in many documents at once

Information that answers a single question is rarely in one file. It is spread across policies, reports, manuals and research papers.

Policies and guidelines

Rules, eligibility criteria, approval routes

Academic and research documents

Papers, syllabi, standards, technical notes

Reports and manuals

Annual figures, procedures, operating handbooks

WHY THIS IS HARD

Manual search is slow
Reading four documents to find one line

Relevant details get missed
The answer may sit in the file you skipped

AI can answer without evidence
Fluent, confident and sometimes invented

The source cannot be checked
No way to confirm where an answer came from

PROPOSED SOLUTION

A Multi-Document RAG system with citations

1. **Accept multiple documents as one knowledge source**

   PDFs, reports, notes — mixed formats in a single corpus

2. **Understand and process document content**

   Text extraction, cleaning and page-aware segmentation

3. **Retrieve only what is relevant to the query**

   Semantic and keyword search over every document at once

4. **Generate the answer from that retrieved context**

   The model is restricted to the evidence it was given

5. **Attach document and page citations**

   Every sentence carries a marker back to its source

CORE IDEA

Retrieve first.
Generate second.

The relevant passages are found before the model writes anything. It never answers from memory — it answers from the text it was just shown.

Built-in refusal

When the documents do not contain the answer, the system says so instead of inventing one.

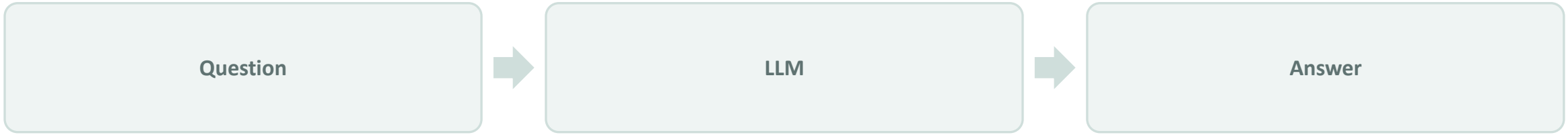
UNDERSTANDING RAG

Retrieval-Augmented Generation

Instead of letting the model answer directly from what it memorised during training, RAG searches the provided documents first and answers only from what it finds.

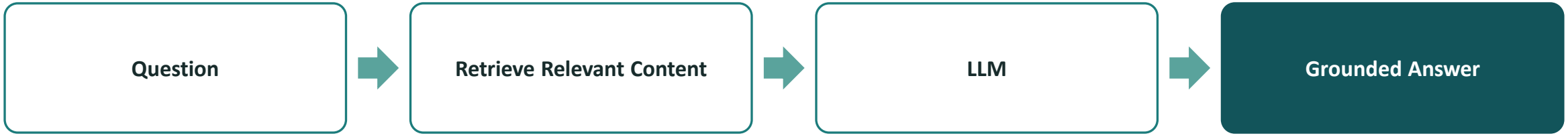
TRADITIONAL APPROACH

answers from memory — no source to check



OUR RAG APPROACH

answers from the documents — every claim traceable



Main benefit: the answer is generated from information retrieved out of the provided documents, so every claim can be checked against the exact page it came from.

SYSTEM METHODOLOGY

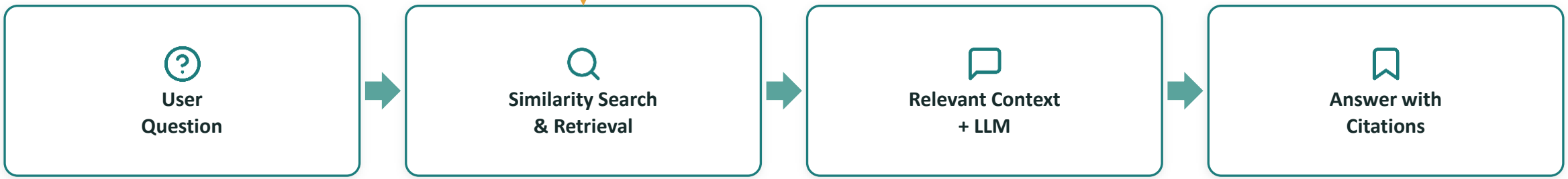
How the system will work

DOCUMENT PROCESSING PIPELINE



the knowledge base built above is what every query searches

QUESTION ANSWERING PIPELINE



Pipeline 1 runs once
when a document set is added or changed

Pipeline 2 runs every time
a question is asked, in a couple of seconds

Citations come from metadata
the document and page stored beside each vector

STAGE ONE

From documents to a searchable knowledge base

Text Extraction

1

Pull usable text out of every uploaded document, including tables. Scanned pages are read with OCR so image-only PDFs are not lost.

Chunking

2

Split long documents into smaller passages at sentence boundaries, each one small enough to be quoted precisely.

Embeddings

3

Convert each passage into a numerical vector that represents its meaning, so related text sits close together.

Vector Storage

4

Store every vector with its metadata — document name, page number and section — which is what makes a citation possible.

Result: one searchable knowledge base built from many documents — and every passage in it still knows exactly where it came from.

STAGE TWO

From a question to a cited answer

1. **User asks a question**

   In plain language, as they would ask a person

2. **Semantic search**

   The query is embedded and compared against every stored passage

3. **Context retrieval**

   The best passages are selected — from one document or from several

4. **Answer generation**

   Only those passages are given to the LLM, with instructions to use nothing else

5. **Citations attached**

   Each sentence is linked back to the document and page that supports it

WORKED EXAMPLE

“What are the eligibility requirements?”

RETRIEVED PASSAGES

- S1 admissions-policy.pdf · p. 3
- S2 programme-handbook.pdf · p. 11
- S3 fee-circular.pdf · p. 2

Answer drawn from 3 documents

cited as S1, S2 and S3 inside the text

INPUT AND EXPECTED OUTPUT

What goes in, what comes out

INPUT

- Document 1 · policy
- Document 2 · handbook
- Document 3 · circular

USER QUESTION

“What are the placement eligibility requirements?”

EXPECTED OUTPUT

GENERATED ANSWER

A student must have cleared all backlogs and hold an aggregate of 60 percent to register for placement.

SUPPORTING CITATIONS

- S1 admissions-policy.pdf · p. 3
- S2 programme-handbook.pdf · p. 11

Answer + Evidence + Verification

TOOLS AND IMPLEMENTATION

Technology stack

Programming language	Python 3.11
Document processing	pdfplumber, pypdf
Chunking	Page-aware, sentence-boundary segmentation
Embeddings	nomic-embed-text (runs locally)
Vector database	ChromaDB / pgVector
Retrieval	Semantic Search (Cosine Similarity)
Generation	Qwen 2.5 7B served locally by Ollama
Interface	React / Vite web application
Evaluation	Groundedness and citation-accuracy metrics

FINAL IMPLEMENTATION

A web application where a user can:

- ✓ Upload multiple documents
- ✓ Ask questions in natural language
- ✓ Receive context-based answers
- ✓ View the supporting citations

Runs entirely offline

The language model and the embedding model both run on the local machine through Ollama. No API keys, no cloud calls and no document ever leaves the laptop.

EXPECTED OUTCOME

What the system will deliver

Faster retrieval

Seconds instead of manual reading

Across documents

One question, the whole corpus

Grounded answers

Generated from retrieved text only

Source-supported

Document and page on every claim

Verifiable

Any answer can be checked at source

THE COMPLETE JOURNEY

Documents → Processing → Embeddings → Retrieval → LLM → **Answer + Citations**

FINAL GOAL

To build a reliable GenAI system that helps users obtain accurate and verifiable information from multiple documents.